A BEGINNER'S GUIDE TO MACHINE-LEARNING ERUPTION FORECASTING

How a computer learns to warn us before a volcano erupts

Before an eruption, magma and gas moving underground make the ground shiver in particular ways. A seismometer records that shiver. This project turns months of those recordings into one simple, explainable answer, refreshed every 10 minutes.



Ground motion recorded by a seismometer (sketch) — Eruption

The one question the computer answers

“Looking at the last 2 days of ground shaking, how likely is the volcano to erupt in the next 2 days?”

Look-back window · 2 days
How far back each example looks. The model only ever sees one 2-day slice of shaking at a time.

Forecast horizon · 2 days
How far ahead the warning reaches. A “yes” means an eruption is expected within this horizon.

Output · a probability from 0 to 1
Not a yes/no oracle: a number plus how uncertain it is, and a breakdown of why.

The forecasting workflow, stage by stage

Each stage takes the previous stage's output and hands a smaller, smarter result to the next. Click a stage to open its page.

01 · Waveform data
Read the raw shaking
Load one day of seismometer

02 · Tremor
Summarise every 10 minutes
Shrink millions of points into

03 · Labels
Mark the answers in the past
Slice history into 2-day

04 · Features
Describe windows with numbers
Compute hundreds of

recordings at a time, from a local archive or a web service.

→ waveforms

RSAM, DSAR and entropy: how loud, what pitch, how orderly.

→ tremor table

windows and tag each 1 if an eruption followed, 0 if not.

→ labelled windows

statistics per window, then keep the 20 that best separate calm from unrest.

→ feature matrix

05 · Training

Teach a panel of models

Train 4 kinds of tree-based model, 25 runs each: an ensemble of 100 forecasters.

→ trained ensemble

06 · Forecasting

Forecast new days

Every 10 minutes, all 100 models score the latest window and their votes are averaged.

→ probability curve

07 · Evaluation

Grade against reality

Compare forecasts with eruptions the models never saw. Recall, precision, ROC and more.

→ scorecards

08 · Explanation

Show the reasoning

SHAP splits each forecast into the push of every feature, so scientists can check it makes sense.

→ explanation plots

Preparing data: stages 01–04 Machine learning: stages 05–08

Learn from the past, then test on the future

The year of data is split in time. Models study the earlier months, where we already know when eruptions happened, and are then tested on later weeks they have never seen. Testing on unseen data is the only honest way to know whether a forecast would have worked.



Training period · known eruptions (white ticks) teach the models

Forecast period · about four weeks of unseen data with held-out eruptions

Dates from the example set-up for station OJN; any split can be used.

What goes in

- **Seismic recordings** from one station, e.g. `VG.OJN.00.EHZ`, stored as miniSEED files.
- **A list of eruption dates** that really happened. These are the "answers" the models learn from.
- **A few settings**: date ranges, which models to train, how many repeat runs.

What comes out

- **A forecast curve**: eruption probability every 10 minutes, with an uncertainty band.
- **Scorecards** showing how often the models caught real eruptions and how often they cried wolf.
- **Explanation plots** naming which tremor patterns pushed each forecast up or down.

Settings used throughout this guide

One example set-up runs through every page, so the numbers can be followed from stage to stage.

STATION
VG.OJN.00.EHZ
vertical-component seismometer

TREMOR SAMPLING
Every 10 minutes
144 samples a day

FREQUENCY BANDS
5 bands, 0.01–16 Hz
RSAM, DSAR and entropy

LOOK-BACK WINDOW
2 days of tremor
what each forecast is based on

FORECAST HORIZON
2 days ahead
how far the warning reaches

WINDOW STEP
6 h · 10 min
training · forecasting

ENSEMBLE
4 model types × 25 runs
100 forecasters

ALERT THRESHOLD
0.7 probability
above this, an alarm is raised

Start here

01 · Seismic waveform data →

STAGE 01 / 08 SEISMIC WAVEFORM DATA

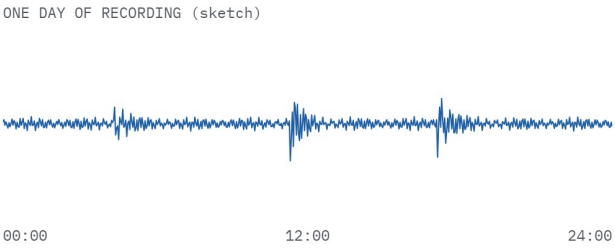
Getting the raw shaking off the volcano

Everything starts with a seismometer: an extremely sensitive motion sensor placed on the volcano. It records how the ground moves, many times every second, around the clock. Stage 01 finds those recordings and loads them, **one calendar day at a time**, so the next stage can measure them.

The idea in plain words

Think of a seismometer as a **microphone for the ground**. Instead of sound, it hears vibrations: passing trucks, ocean waves, earthquakes, and, near a volcano, the rumble of magma and gas pushing through rock.

The recording is called a **waveform**: a long list of ground-motion values with a timestamp each. A single day contains millions of values, saved in a standard file format called **miniSEED**.



Up and down = the ground moving. Bigger swings = stronger shaking.

Every sensor has a SEED code

Seismologists identify each data stream with four codes joined by dots, following the international SEED convention: network, station, location and channel. Everything in this guide comes from one stream, VG.OJN.00.EHZ.

VG
Network

The organisation or network that runs the station.

OJN
Station

The physical site on the volcano where the sensor sits.

00
Location

Tells sensors apart when a site has more than one. Often empty or “00”.

EHZ
Channel

Band, instrument and component: E = short-period at a high sample rate, H = high-gain seismometer, Z = vertical motion.

Two places the data can come from

Either source can be used. Both deliver the same thing, one day of continuous waveform for the station, so the later stages don't depend on where it came from.

SDS · a folder on your disk

The **SeisComP Data Structure** is a standard way of filing recordings: one file per station, channel and day, in nested folders. If your observatory already archives data this way, the forecasting system simply reads the right file. No internet

FDSN · a web service

Many data centres share recordings over the internet through the **FDSN** web-service standard (the default is IRIS). The system downloads a day, then **saves a copy in SDS layout** so the next run reads from disk instead.

needed.

HOW A FILE PATH IS BUILT

`D:\Data\OJN\` `2025\` `VG\OJN\` `EHZ.D\`
`VG.OJN.00.EHZ.D.2025.015`

archive root → year → network\station → channel.type → file. The last number is the **Julian day**, the day of the year: 015 is 15 January.

1. Ask for one day, e.g. 15 Jan 2025.
2. **Downloaded before?** Use the local copy.
3. **Not yet?** Download it from the data centre.
4. Store a local copy in SDS layout for next time.

Once every day has been downloaded, the local copy is an ordinary SDS archive.

What happens to each day

1

Pick the day

Work through the period from the first to the last day. Several days can be processed at the same time.

2

Find the file

Locate the day-file from the station code and date, or request it from the data centre.

3

Read it

The miniSEED file is decoded into one or more traces: continuous runs of samples.

4

Stitch the gaps

Recordings often come in pieces. They are merged into one trace, and short gaps are filled by interpolation.

5

Hand it on

The clean day goes on to stage 02. A day with no data is skipped.

Words to know

`Seismogram (waveform)`
The raw record of ground motion over time.

`miniSEED`
The standard compact file format for waveforms.

`Trace`
One continuous segment of a seismogram from a single channel.

`Julian day`
Day number within the year, 001 to 365 (366 in leap years).

Back
← Overview

Next
02 · Tremor calculation →

STAGE 02 / 08 TREMOR CALCULATION

Turning raw waves into tremor numbers

A day of raw recording holds millions of values: far too many, and too noisy, to learn from directly. Stage 02 boils each day down to **one value every 10 minutes** that answers three questions: how strong is the shaking, has its pitch shifted, and how orderly is it?

IN · per day

Millions

of raw ground-motion samples from stage 01

→

OUT · per day

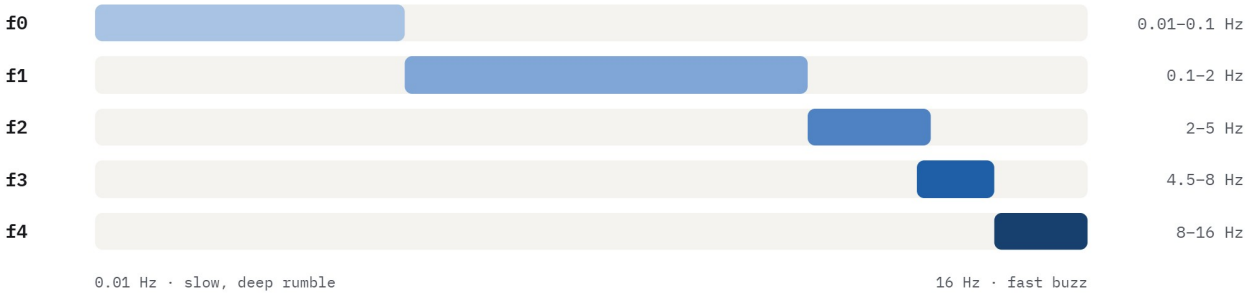
144 × 10

one value every 10 minutes for each of ten tremor series

STEP 1

Split the signal into five frequency bands

Like the equaliser on a stereo that separates bass, mids and treble, a **band-pass filter** keeps only the vibrations within a range of frequencies (measured in hertz, Hz: vibrations per second). Different volcanic processes tend to shake at different frequencies, so measuring each band separately keeps those signals apart.



Bars drawn on a logarithmic frequency scale. Bands are the project defaults and can be changed.

STEP 2

Divide each day into 10-minute intervals

24 hours ÷ 10 minutes = **144 intervals a day**. Every measurement below is computed once per interval, giving one sample of each tremor series every 10 minutes.



STEP 3

Take three measurements per interval

"How strong is the shaking?"

RSAM

Real-time Seismic Amplitude Measurement

```
mean( |signal in band| )
```

The average size of ground motion in one band during the interval. When magma and gas are on the move, tremor often grows and RSAM climbs. The classic volcano-monitoring number.

`RSAM f0` `RSAM f1` `RSAM f2` `RSAM f3` `RSAM f4`

"Has the pitch shifted?"

DSAR

Displacement Seismic Amplitude Ratio

```
mean|lower band| ÷ mean|next band up|
```

The signal is first converted from velocity to displacement, then two neighbouring bands are compared. A rising ratio means energy is moving toward lower frequencies, a change linked to fluids migrating and pressure building.

`DSAR f0/f1` `DSAR f1/f2` `DSAR f2/f3` `DSAR f3/f4`

"Is it orderly or random?"

Entropy

Shannon entropy

$$H = -\Sigma\ p(x) \cdot \log_2 p(x)$$

A measure of unpredictability borrowed from information theory. Low entropy means a more regular, repetitive signal; high means noise-like. Very quiet intervals (too little energy) are left blank.

`Entropy`

STEP 4

Clean up, then stack the days together

Remove spikes

A single huge outlier (e.g. a nearby knock on the sensor) is removed so it can't dominate.

Drop broken days

If less than 30% of a day has usable data, the day is discarded rather than guessed.

Save daily files

Each day is stored on its own, so updating the record only processes new days.

Merge

All days are joined into one continuous tremor time series: 144 × 365 = 52,560 samples per year.

The result: ten tremor time series

Ten series sampled every 10 minutes. The example set-up uses five of them (highlighted) for training; the others stay available for experiments.

Time (UTC)	RSAM f0	RSAM f1	RSAM f2	RSAM f3	RSAM f4	DSAR f0/f1	DSAR f1/f2	DSAR f2/f3	DSAR f3/f4	Entropy
2025-03-18 00:00	·	·	·	·	·	·	·	·	·	·
2025-03-18 00:10	·	·	·	·	·	·	·	·	·	·
2025-03-18 00:20	·	·	·	·	·	·	·	·	·	·

… one sample every 10 minutes for the whole period

Words to know

Tremor

Continuous, low-level shaking of a volcano, as opposed to a single earthquake.

Band-pass filter

Keeps vibrations between two frequencies and removes the rest.

Hertz (Hz)

Vibrations per second. Low Hz = slow swaying, high Hz = fast buzzing.

Displacement

How far the ground moved, obtained by integrating velocity over time.

Previous
← 01 · Waveform data

Next
03 · Labelling →

STAGE 03 / 08 LABELLING

Teaching by example: marking the answers in the past

Machine learning needs examples that already have the right answer attached. Because we know when past eruptions happened, we can cut the tremor history into short windows and tag each one: **1** if an eruption followed soon after, **0** if it didn't.

The idea in plain words

Picture a stack of **flashcards**. The front of each card shows two days of tremor. The back says either “an eruption came within the next two days” or “nothing happened”.

Stage 03 makes the cards. Stage 04 describes the front of each card in numbers. Stage 05 studies the whole stack until the models can guess the back from the front.

CARD FRONT



Tremor for 2 days ending at a moment in time

CARD BACK

1

Label 1: an eruption followed within the next 2 days

RULE 1 · WHICH MOMENTS COUNT AS “BEFORE AN ERUPTION”

The 2 days before an eruption are marked 1

With a **forecast horizon** of 2 days, every moment from two days before the eruption through the eruption day is positive. Everything else is 0. Example: the eruption on 20 March 2025.



A setting decides whether the eruption day is counted inside the 2 days (2 positive days) or added on top (3 positive days, the default shown here).

RULE 2 · HOW THE CARDS ARE CUT

Overlapping 2-day windows, a new one every 6 hours

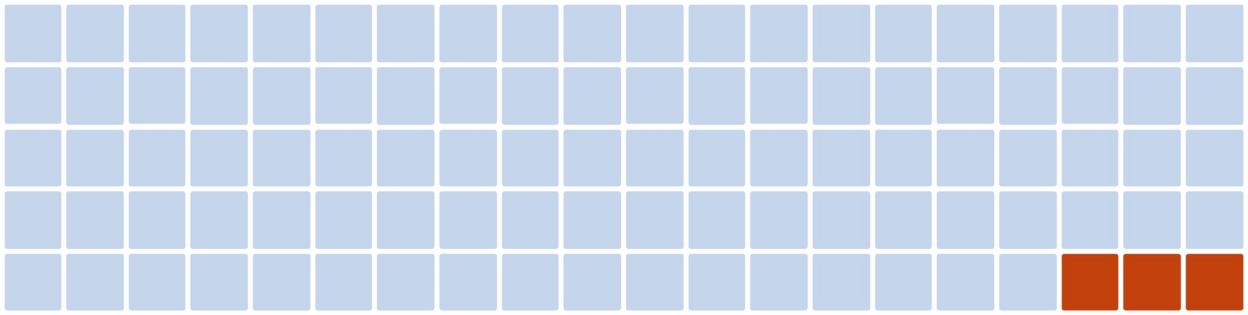
Each window covers a **2-day look-back** of tremor and takes the label of the moment it **ends**. The window then slides forward by a fixed **step**: 6 hours when training (4 cards a day), 10 minutes when forecasting (144 a day). Neighbouring windows share most of their data, which gives many examples from a short record.





A catch: eruptions are rare

In the example training period (1 Jan to 26 Jul 2025) there are 2 eruptions, each giving 3 positive days: about 6 of 207 days, roughly **3 in every 100** cards. A lazy model could say “no eruption” every time and be right 97% of the time while being useless. Stage 05 fixes this by rebalancing.



Two ways to build labels

`Standard labelling`

One timeline over the whole date range, windows stepping evenly from start to end. Simple and the default.

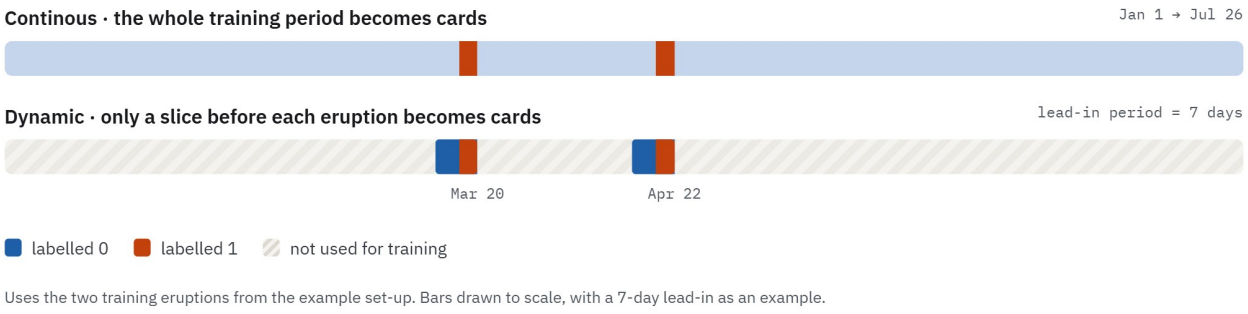
`Dynamic labelling`

Labels only a short slice before each eruption and skips the long quiet stretches in between. Explained step by step below.

OPTION · DYNAMIC LABELLING

Dynamic labelling: one slice per eruption

Standard labelling covers **every** window in the training period, so months of quiet tremor produce thousands of 0-cards. Dynamic labelling does something different: for each eruption it cuts out only a short **lead-in period** leading up to it and labels just that slice. The days between slices are left out of the card stack entirely.



Uses the two training eruptions from the example set-up. Bars drawn to scale, with a 7-day lead-in as an example.

How it is built, in three phases

Example: two eruptions only three days apart, A on 20 March and B on 23 March, with a 7-day lead-in period and a 2-day forecast horizon. Each square is one day; in reality each day holds 4 windows (one every 6 hours).



1

Cut one slice per eruption

Only eruptions inside the training period are used. For each one, windows are laid out every 6 hours from the start of its lead-in period up to the eruption date. Every window starts as 0. The dashed red outline marks each slice's future positive zone (the eruption day plus the 2 days before it), which only becomes 1 in phase 3.

2

Merge and remove duplicates

The slices are joined into one table, sorted by time. When eruptions are close, slices overlap (16–20 March above), so a timestamp would appear twice. Each is kept once and the windows are renumbered 0, 1, 2…

3

Mark positives per eruption

For each eruption in turn, windows from 2 days before through the end of the eruption day are set to 1. Doing this **after** the merge means one eruption's 1s can never be overwritten by the other slice's 0s.

Why use it

- 1s make up a much bigger share of the cards, so the data is far less lopsided before resampling even starts.
- The models compare “just before an eruption” with “a few days earlier at the same volcano”: a sharper contrast to learn from.
- Eruptions that follow each other closely are handled cleanly.

What to watch out for

- The models never see the long quiet periods, but the forecast still runs on them. Check the false-alarm rate in stage 07.
- The lead-in period must be longer than the 2-day warning zone, or there are no 0s to learn from.
- The labelled period is set by the eruption dates themselves, not by a chosen start and end date.

Words to know

`Label`
The known answer for a window: 1 or 0.

`Positive / negative`
Windows labelled 1 / windows labelled 0.

`Look-back window`
How many days of tremor each window covers (2).

`Step`
How far the window slides between examples (6 h in training).

`Class imbalance`
When one answer (0) vastly outnumbers the other (1).

`Lead-in period`
Dynamic labelling only: how many days before each eruption its slice starts.

Previous
← 02 · Tremor

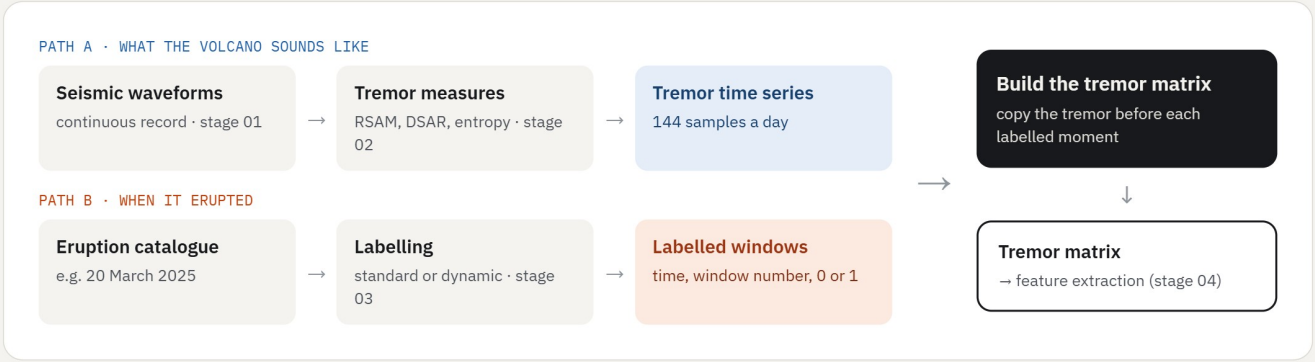
Next
03b · Tremor matrix →

STAGE 03B TREMOR MATRIX

The tremor matrix: the tremor just before each labelled moment

So far we have two separate records: the **tremor time series**, what the volcano sounded like every 10 minutes, and the **labelled windows**, a list of moments each tagged 0 or 1. The tremor matrix joins them. For every labelled moment it looks back 2 days, copies that stretch of tremor, tags it with the window's number, and stacks everything into one long table.

Two paths meet here



The three settings that shape it

Look-back window · 2 days
How far back to look from each labelled moment.

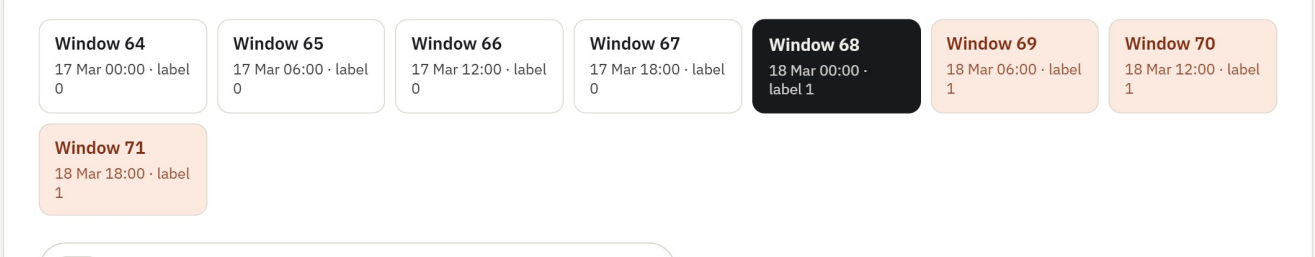
Sampling · every 10 minutes
So each window should hold 2 × 144 = **288 samples**.

Required completeness · 100%
Share of those 288 samples that must exist. By default no data gap at all is tolerated.

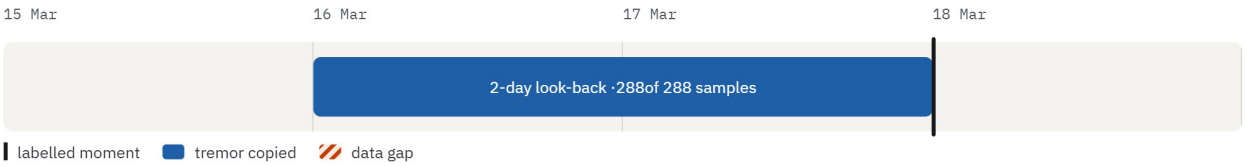
TRY IT

Pick a labelled window and watch its tremor

Example labels from stage 03: labelling starts 1 March 2025 (window 0 ends at 1 Mar 00:00), one window every 6 hours, eruption on 20 March, so windows ending from 18 March 00:00 are labelled 1.



Simulate a 4-hour data gap at the station (16 Mar 06:00 – 10:00)



Window 68 ends at **2025-03-18 00:00**. Its tremor runs from **2025-03-16 00:00** to **2025-03-17 23:50**: the 48 hours just **before** the labelled moment, never after it.

samples found
288 / 288

completeness
100%

Kept
Every expected sample exists, so all 288 are copied and tagged with window 68.

Window	Time (UTC)	RSAM f2	RSAM f3	RSAM f4	DSAR f3/f4	Entropy
68	2025-03-16 00:00	18.2	9.6	5.1	1.88	0.81
68	2025-03-16 00:10	18.9	9.9	5.3	1.86	0.80
⋮ 285 more samples, all tagged window 68						
68	2025-03-17 23:50	20.8	11.1	5.9	1.85	0.79

Example values, for shape only. Every copied sample is tagged with its window number.

STACK EVERY WINDOW

The result: the tremor matrix

All complete windows are placed one after another, in time order. The matrix doesn't get wider with more windows, only longer.

Window 64	288 samples
Window 65	288 samples
Window 66	288 samples
Window 67	288 samples
Window 68	288 samples
Window 69	288 samples
Window 70	288 samples
Window 71	288 samples

8 windows × 288 samples = 2,304 rows

Every training day adds 4 windows × 288 = 1,152 rows, so a few months of record quickly reaches hundreds of thousands of rows.

Each row holds: window number, time, and the five tremor measures used for training (RSAM f2, f3, f4, DSAR f3/f4, entropy).

Windows overlap, and that's fine

Labelled moments are 6 hours apart but each look-back is 48 hours long, so neighbouring windows share **252 of their 288** samples. Pick window 66 then 67 and watch the look-back slide by one eighth of its length. A single 10-minute sample can appear in up to 8 windows.

Where did the label go?

It is **not** copied into the matrix. It stays with the labelled windows, and the two are matched by window number when features are extracted and the models are trained.

The first window needs earlier tremor

Window 0 (1 Mar 00:00) looks back to 27 Feb. If the tremor record doesn't reach that far, the window is incomplete and dropped, just like a data gap. The matrix only covers the period where tremor exists.

Why stack the windows?

The same procedure for

Works with dynamic labels too

Keeping each window as one block of 288 consecutive samples, tagged with its number and in time order, lets stage 04 treat every block as its own short time series and summarise it into a single row of features.

forecasting

In stage 06 a window ends every 10 minutes and its outcome is still unknown. The tremor is copied exactly the same way, just for many more windows: 144 a day.

The matrix only needs each window's end time and number, so windows from dynamic labelling each get their own 2-day look-back in exactly the same way.

Previous
← 03 · Labelling

Next
04 · Feature extraction →

STAGE 04 / 08 FEATURE EXTRACTION

Describing each window with numbers

A scientist can glance at two days of tremor and say “that's rising and getting spiky”. A model can't look at a curve, so we describe every window with a list of measurements called **features**, such as its average, its peak and its trend. Then we keep only the few that actually help tell calm from unrest.

From one window to 20 numbers

A · SLICE
288
tremor samples per series per window (2 days × 144)

→

B · DESCRIBE
~700
statistics per tremor series, computed automatically

→

C · CANDIDATES
~3,500
for 5 tremor series: one long row per window

→

D · SELECT
20
most informative features kept per model

STEP A

Cut the tremor series into labelled windows

For every labelled moment from stage 03, the 2 days of tremor that end at that moment are copied out and tagged with the window's number: the tremor matrix. Windows with missing data are rejected so the model never learns from half-empty records. See step by step how the tremor matrix is built →

STEP B

Measure everything, automatically

A standard time-series toolbox computes hundreds of statistics for each tremor series in each window. A few families are switched off because they add little here: aggregated linear trends, time-based linear trends and window length.

Size
mean, median, maximum, quantiles. How big was the tremor?

Spread
standard deviation, variance. Was it steady or jumpy?

Change
mean change, absolute sum of changes. Was it climbing?

Rhythm
autocorrelation, Fourier (spectral) coefficients. Did patterns repeat?

Shape
skewness, kurtosis, peaks. Were there sudden spikes?

Complexity
sample entropy, complexity estimates. Was it orderly?

HOW TO READ A FEATURE

`RSAM f3` · `quantile` · `q = 0.9`

Which column
RSAM in the 4.5–8 Hz band

Which calculation
a quantile (a percentile)

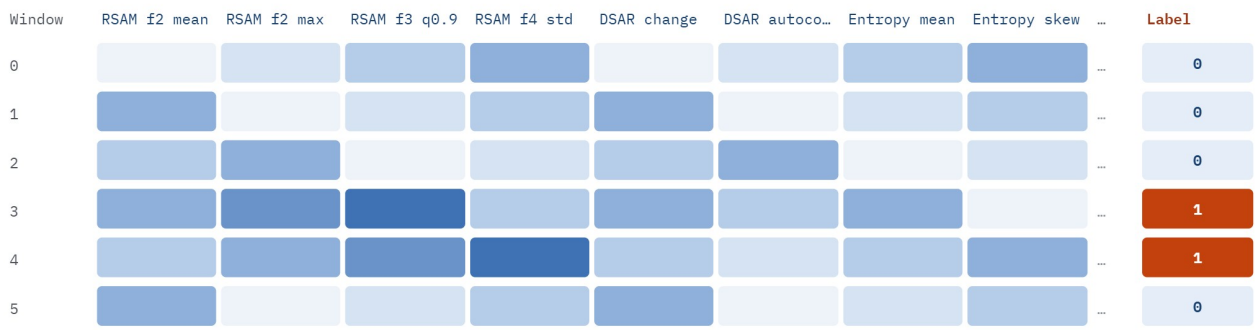
With which setting
q = 0.9, the 90th percentile

In words: “the level that RSAM in band f3 stayed below 90% of the time during these 2 days”. A high value means the window held a lot of strong tremor.

THE RESULT

The feature matrix: one row per flashcard

Rows are windows, columns are features, and the label from stage 03 rides along on the right. This table, a spreadsheet of numbers with an answer column, is exactly what machine-learning models are built to read.



Schematic. Shade stands for a value (darker = larger); feature names are shortened.

STEP D

Keep the 20 features that matter

Thousands of features would let a model memorise noise. So each feature is tested: do its values differ between 1-windows and 0-windows more than chance would explain? Features that pass are ranked and the top 20 kept.

`Significance test (default)`

A statistical test per feature gives a p-value. The **FDR** (false discovery rate) filter controls how many false hits slip through when testing thousands of features at once.

`Random-forest importance`

Trains a quick forest and measures how much its accuracy drops when each feature is shuffled (permutation importance).

Important timing detail: selection is not done once for everyone. It runs inside every training run in stage 05, after that run has rebalanced its data. Each of the 100 models can end up with a slightly different set of 20 features. That is why explanations in stage 08 are computed per run.

Words to know

`Feature`
One number describing a window, such as its mean or 90th percentile.

`Time-series statistics`
Standard summary measures of a signal: level, spread, trend, rhythm and shape.

`Feature selection`
Keeping only features that carry real signal about the label.

`Overfitting`
Memorising quirks of the training data instead of general patterns.

Previous
← 03b · Tremor matrix

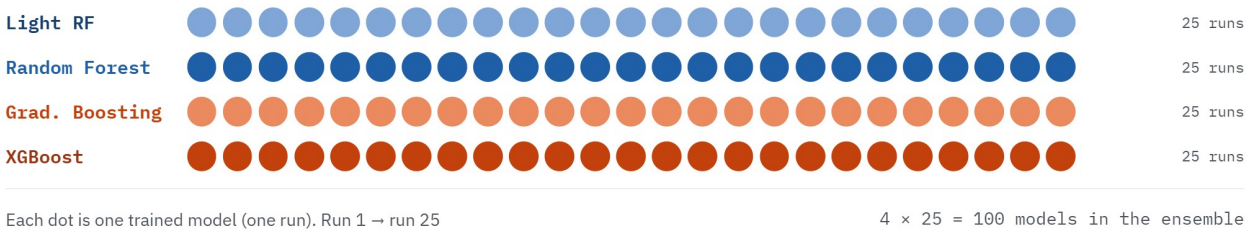
Next
05 · Training →

STAGE 05 / 08 TRAINING

Training a panel of 100 forecasters

Rather than trusting one model, the forecasting system trains **4 kinds of tree-based model**, each **25 times** with a different random draw of the data. That makes an ensemble of 100 models. Each studied a slightly different shuffle of history, so when they agree we can be more confident, and when they disagree we know the forecast is uncertain.

The panel: 4 model types × 25 runs



Each dot is one trained model (one run). Run 1 → run 25

4 × 25 = 100 models in the ensemble

Random Forest

Hundreds of decision trees, each grown on a random sample of the data, then they vote. Like asking a crowd.

Light Random Forest

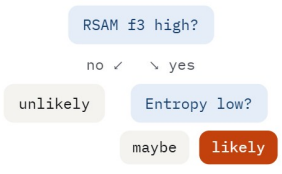
A lighter random forest with fewer settings to search, so it trains quickly.

Gradient Boosting

Trees built one after another, each one fixing the mistakes of those before it.

XGBoost

A fast, carefully regularised version of gradient boosting, widely used in practice.



What is a decision tree? A flowchart of yes/no questions about the features, learned from the data. All four model types are built from many such trees. That is also what lets stage 08 explain them exactly. (Simplified example.)

What happens in one training run

The same four steps repeat for run 1 to 25. Each run draws the data differently, so the runs differ from one another, yet every run can be repeated exactly. Several runs are computed at the same time.

1

Rebalance the cards

Eruption cards are rare, so a random subset of normal cards is set aside (**undersampling**) until there are 3 eruption cards for every 4 normal ones. Each run sets aside a different subset.

2

Pick 20 features

Run the feature selector from stage 04 on this run's balanced cards and keep the top 20.

3

Tune each model type

A **settings search** tries combinations of settings (tree depth, number of trees…) and scores each with 5 rounds of cross-validation (below). The combination with the best recall

4

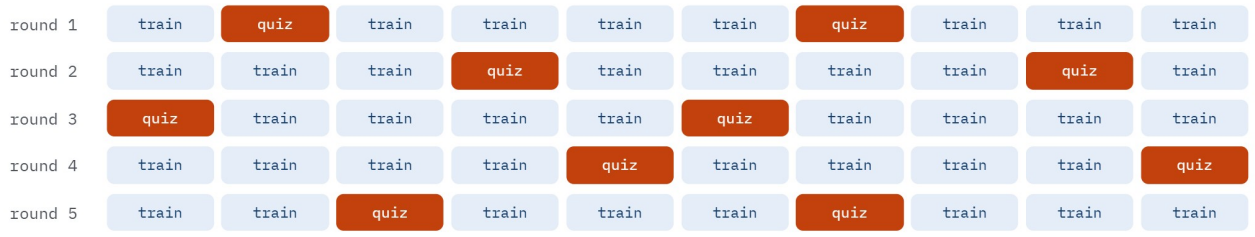
Save the winner

The best model of this run is kept, together with the 20 features it uses.

wins.

Cross-validation: a practice exam inside training

To choose settings fairly, a model is never graded on cards it trained on. The cards are shuffled and a random portion is held back as a quiz, keeping the same mix of 1s and 0s in both parts (**stratified**). This is repeated 5 times with a different random quiz each round, and the scores are averaged.



TARGET · PROBABILITY OF DETECTION

Why detection comes first

Recall, the probability of detection, asks: of all the moments before real eruptions, how many did the model flag? Settings are chosen to maximise it because **missing an eruption is far worse than a false alarm**. A false alarm costs effort; a miss can cost lives.

ENSEMBLE SPREAD

Why 25 runs?

A single model can be lucky or unlucky with the slice of history it happened to see. Repeating the training 25 times with different random draws shows how much the answer depends on chance: when the runs agree the forecast is robust, when they scatter it is uncertain. That spread becomes the uncertainty band in stage 06.

Words to know

`Classifier`
A model that sorts examples into classes, here 1 or 0.

`Training run`
One complete training with its own random draw of the data.

`Ensemble`
A group of models whose answers are combined.

`Model settings`
Choices fixed before training, such as how deep each tree may grow.

`Undersampling`
Dropping some majority-class examples to even out the classes.

Previous
← 04 · Features

Next
06 · Forecasting →

STAGE 06 / 08 FORECASTING

Forecasting: all 100 models vote every 10 minutes

Now the trained panel faces days it has never seen. Every 10 minutes the forecasting system builds the same kind of 2-day window, describes it with the same features, and asks every model the same question. The answers are averaged into one **eruption probability**, plus a measure of how much the models disagree.

The same recipe, run on new data

1

Windows every 10 min

A new 2-day window ends every 10 minutes: 144 forecasts a day, so the curve is smooth.

2

Same features

The same statistics describe each new window. No new selection: each model reads the 20 features it was trained on.

3

100 probabilities

Each model returns a number from 0 to 1: its belief that an eruption comes within 2 days.

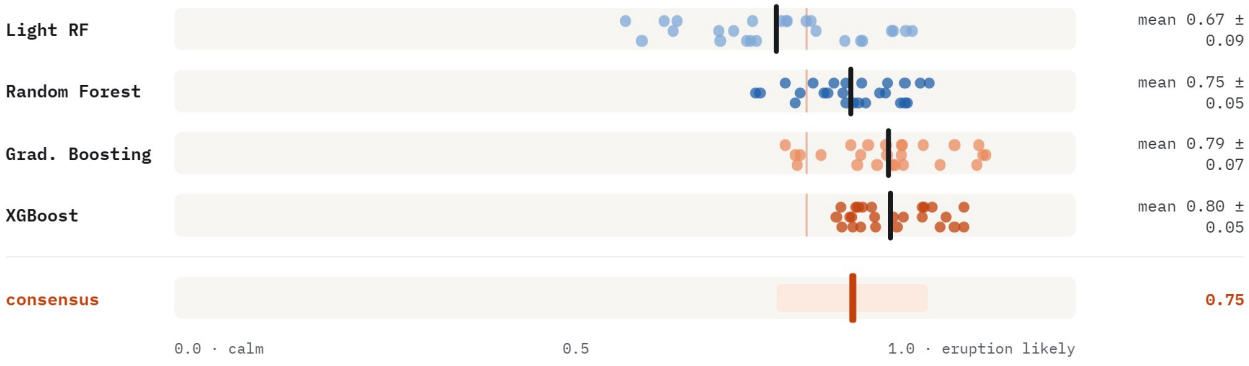
4

Combine

Average within each model type, then across types, giving the **consensus**. The spread becomes the uncertainty.

How 100 votes become one number

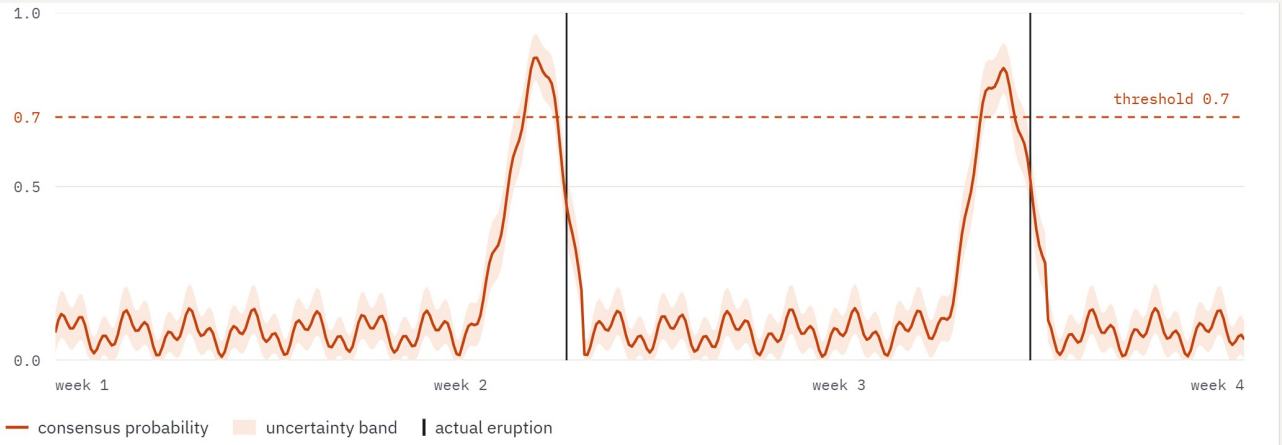
One forecast moment, illustrated. Dots are the 25 runs of each model type, placed by the probability they gave. Spread-out dots mean the runs disagree.



Illustrative values, not real output. The faint orange line marks the 0.7 alert threshold used in the example plot.

Reading the forecast plot

String the consensus for every 10-minute step together and you get the main product of the forecasting system: a probability curve through time.



Schematic of the plot's layout. Real curves are noisier.

A good forecast

Stays low during quiet periods, then climbs above the threshold in the days **before** each eruption line, not after it.

The threshold

A line you choose (0.7 in the example). Crossing it means “alert”. Lower = more warnings and more false alarms.

The band

Wide band = the 100 models disagree. Treat a high value with a wide band more cautiously.

The results table

output	meaning
`Consensus eruption probability`	Average of the four model types' probabilities.
`Consensus uncertainty`	Spread (standard deviation) pooled across all model types and runs.
`Consensus confidence`	1 – normalised uncertainty. Close to 1 = the models agree.
`Consensus alarm (yes/no)`	The yes/no call (1 or 0) made from the consensus probability.
`Per model type`	The same four values for each model type separately.

Previous
← 05 · Training

Next
07 · Evaluation →

STAGE 07 / 08 EVALUATION

Grading the forecast against what really happened

A forecast is only useful if it would have worked. Because the forecast period contains eruptions the models never saw, we can line up every prediction with the true label and count hits, misses and false alarms, for each of the 100 models.

Four possible outcomes

Every forecast window falls into one box of this grid, called a **confusion matrix**. All the scores below are built by counting the boxes.

	eruption followed	no eruption
alarm	**Successful warning** — True positive: warned, and it erupted. The goal.	**False alarm** — False positive: an alarm with no eruption. Costly in effort and trust.
no alarm	**Missed eruption** — False negative: no alarm, yet it erupted. The most dangerous error.	**Correct all-clear** — True negative: quiet, correctly called quiet.

THE ACCURACY TRAP

If 97 of 100 windows are quiet, a model that **always** says “no eruption” is 97% accurate and catches zero eruptions. That's why plain accuracy is reported but never trusted alone.

The scores, in plain words

Recall · detection rate

main target

Of all windows before real eruptions, what share did we flag? **“How many eruptions did we catch?”**

`warnings ÷ (warnings + misses)`

Precision · alarm reliability

Of all the alarms we raised, what share were real? **“Can people trust an alarm?”**

`warnings ÷ (warnings + false alarms)`

F1 score

One number that balances recall and precision. High only when both are high.

`harmonic mean of the two`

Balanced accuracy

Accuracy on eruption windows and on quiet windows, averaged, so the many quiet windows can't dominate.

$\frac{1}{2}$ $(\text{recall}_1 + \text{recall}_0)$

ROC AUC

How well the probabilities rank dangerous windows above calm ones, over every possible threshold. 0.5 = coin toss, 1.0 = perfect.

`area under the ROC curve`

PR AUC · G-mean

Two more rare-event-friendly scores: precision vs recall across thresholds, and a balance of hit rates on both classes.

`precision–recall area · G-mean`

Some scores are also reported “at optimal”, using the threshold that would have worked best. These show the potential of the probabilities, separately from the fixed threshold.

Scored per run, then compared

Every one of the 100 models gets its own scorecard. That shows not just how good a model type is on average, but how **stable** it is from run to run.

1 · PER RUN

100 scorecards

Every score is computed for each run of each model type.

→

2 · PER TYPE

Mean ± spread

Scores are summarised per model type: average and variation across its 25 runs.

→

3 · COMPARE

Ranking + plots

Model types are ranked by recall and drawn side by side.

ROC overlay

All model types' ROC curves on one chart, with a ± spread band.

Metric bars

Each score per model type, mean with error bars.

Run-to-run stability

Violin plots: a narrow shape means a model type gives similar results whatever the run.

Ranking table

Model types sorted from best to worst recall.

Words to know

Held-out data

Data deliberately kept away from training so it can grade the model honestly.

Threshold

The probability above which a window is called “eruption”.

ROC curve

Hit rate plotted against false-alarm rate as the threshold slides from 1 to 0.

Previous
← 06 · Forecasting

Next
08 · Explanation →

STAGE 08 / 08 EXPLANATION · SHAP

Opening the black box: why did the model say that?

A number like “0.75” isn't enough for a volcano observatory to act on. Scientists need to know **what in the tremor** drove it, so they can check it makes physical sense. SHAP splits every forecast into the contribution of each feature: which ones pushed the probability up, which pulled it down, and by how much.

The idea in plain words

Imagine a team wins a prize and must split it fairly by how much each player contributed. Game theory has an exact recipe for that, called **Shapley values**. SHAP (SHapley Additive exPlanations) applies the same recipe to a model: the “prize” is the forecast, the “players” are the 20 features.

The contributions always add up: **starting point + all pushes = the model's output**. Nothing is hidden.

THE ADDITIVE RULE

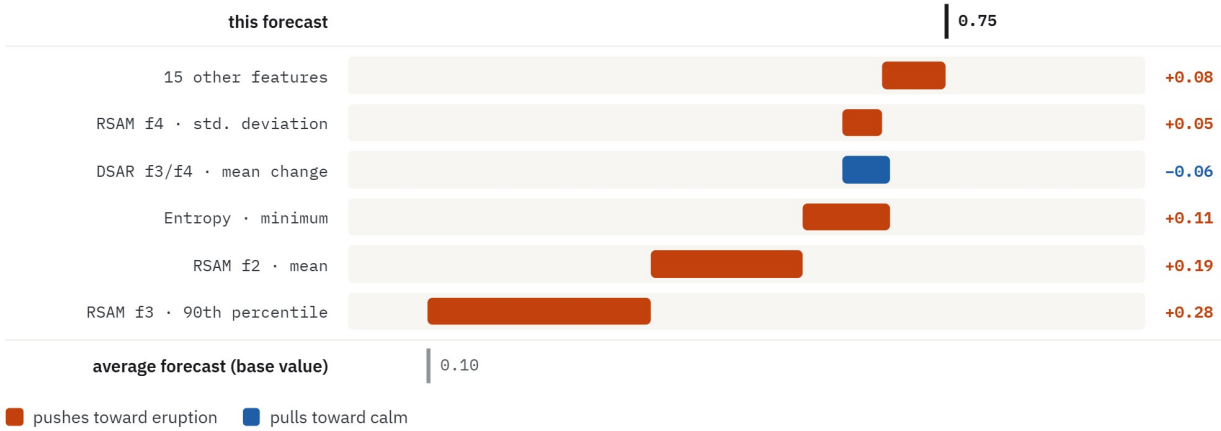
base value + pushes up − pulls down = forecast

The base value is the model's average output across all windows: what it would say knowing nothing about this particular window.

PLOT 1 · ONE FORECAST

The waterfall: one window, step by step

Read from the bottom up. Start at the base value; each feature moves the running total right (toward eruption) or left (toward calm) until you reach the final output at the top.



Illustrative values and feature names, not real output. Real plots may show values on the model's internal score scale instead of probability.

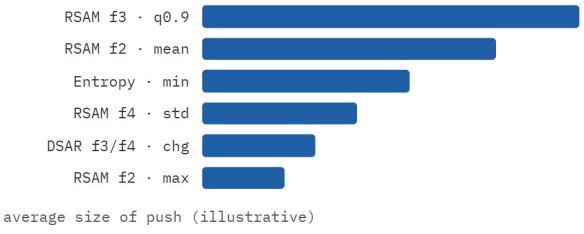
Which window gets a waterfall? There are 100 models × thousands of windows, far too many to plot. So for each eruption day and each model type, the forecasting system finds the single window, and the single run, that gave the **highest eruption probability** that day, and draws the waterfall for exactly that one. It answers: “at the model's most alarmed moment, what was it reacting to?”

PLOTS 2 + 3 · ALL FORECASTS TOGETHER

The big picture: which features matter overall

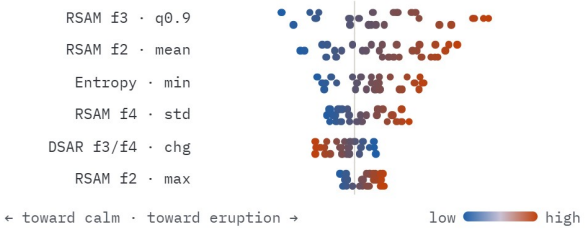
Bar plot: global importance

Average size of each feature's push over every window, ignoring direction. Longest bar = the feature the model leans on most.



Beeswarm: direction and value

Every dot is one window. Left/right = push down/up. Colour = the feature's own value in that window. Orange dots on the right mean "high values of this feature raise the alarm".



Both are drawn per run and as an **aggregate** per model type, stacking all 25 runs together. Because each run may have picked slightly different features (stage 04), a feature that appears near the top across many runs is a robust signal.

Why only tree models

A version of SHAP built for tree-based models gives exact contributions quickly. It covers all four model types used here (Random Forest, Light Random Forest, Gradient Boosting, XGBoost); other kinds of model are not explained.

A word of caution

SHAP explains **the model**, not the volcano. It shows which patterns the model learned to associate with eruptions, which is not proof that those patterns cause eruptions. Its real value is letting volcanologists spot when a model relies on something physically implausible, such as a station glitch.

Words to know

`SHAP value`
How much one feature moved one forecast away from the base value.

`Base value`
The model's average output: the starting point of every explanation.

`Local vs global`
Local = why this one forecast (waterfall). Global = what matters overall (bar, beeswarm).

THE WHOLE JOURNEY

Ground shaking → tremor numbers → labelled windows → features → 100 trained models → a probability every 10 minutes → graded against real eruptions → explained feature by feature.

← Back to the overview

Previous
← 07 · Evaluation